

Q1. Write a Java program to print "Hello" using a thread.

```
class HelloThread extends Thread {
    public void run() {
        System.out.println("Hello");
    }
}

public class HelloThreadDemo {
    public static void main(String[] args) {
        HelloThread t = new HelloThread();
        t.start(); // starts a new thread that runs run()
    }
}
```

Calling `start()` causes the Java Virtual Machine to create a new thread of execution, which then calls the `run()` method, printing Hello on that separate thread rather than on the main thread.

Q2. Create two threads to print even and odd numbers separately.

```
class EvenPrinter extends Thread {
    public void run() {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println("Even: " + i);
        }
    }
}

class OddPrinter extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i += 2) {
            System.out.println("Odd: " + i);
        }
    }
}

public class EvenOddDemo {
    public static void main(String[] args) {
        EvenPrinter evenThread = new EvenPrinter();
```

```

        OddPrinter oddThread = new OddPrinter();

        evenThread.start();
        oddThread.start();
    }
}

```

The EvenPrinter and OddPrinter threads run concurrently and independently, each printing its own sequence of numbers. Because both threads are scheduled independently by the JVM, the even and odd numbers may be printed in an interleaved order rather than strictly sequentially.

Q3. Demonstrate thread sleep and join in Java.

```

class Worker extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Working... step " + i);
            try {
                Thread.sleep(500); // pause for 500 milliseconds
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

public class SleepJoinDemo {
    public static void main(String[] args) throws InterruptedException {
        Worker worker = new Worker();
        worker.start();

        worker.join(); // main thread waits until worker finishes

        System.out.println("Worker thread has finished. Main continues.");
    }
}

```

Thread.sleep(500) pauses the execution of the currently running thread for approximately 500 milliseconds, allowing other threads a chance to run. worker.join() causes the main thread to pause and wait until the worker thread has completely finished executing before continuing, which is useful when the main thread depends on the result of the worker thread's work.